claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_6e519d4b-9e6\agent-a5f3c0df7df8959ac.jsonl`
- SHA-256: `1ad6867272dca25ab3d11d336e9100d12837117146f426826f2227478d6038a6`
- Source modified: `2026-06-22T14:45:16+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_6e519d4b-9e6`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T14:44:22.147Z)

Read C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/__init__.py, base.py, cloud_run.py fully. Document the contract: ComputeJobSpec fields, ComputeResult fields, ComputeBackend.dispatch() signature + behavior, the done-marker / bucket-poll mechanism (poll_until?), and the GoogleCloudRunJobClient (real) vs the INJECTABLE fake client used for offline tests (how is it injected? ? this is how we test item 1 with NO real GPU). What does CloudRunCompute assume about the deployed Cloud Run Job image (env vars, args it passes)? Quote file:line.

2. assistant (2026-06-22T14:44:25.340Z)

I'll read the three files in the compute module to document the contract.

3. user (2026-06-22T14:44:25.845Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
```

```
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38      ) -> Optional[ComputeBackend]:
39          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40          in-process (the default-OFF path).
41
42          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46          """
47          target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48          if target != "cloud_run":
49              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

4. user (2026-06-22T14:44:26.776Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Optional
```

```
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str                          # input (gs://? for cloud; a path/local://
for local)
27          out_uri: str                            # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40          returncode: int
41          outputs_uri: str = ""
42          video_id: str = ""
43          report: dict = field(default_factory=dict)
44
45
46      class ComputeBackend(ABC):
47          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49          execution identically."""
50
51          @abstractmethod
52          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53              """Run ``spec`` to completion and return its ``ComputeResult``."""
54
```

5. user (2026-06-22T14:44:26.970Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
```

```
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14       The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15       so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17       The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18       with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19       owner-verified on the M7 real run.
20       """
21
22       from __future__ import annotations
23
24       import json
25       import logging
26       import os
27       import tempfile
28       import uuid
29       from abc import ABC, abstractmethod
30       from typing import Any, List, Optional
31
32       from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33       from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34       from backend.storage import fetch, parse_uri
35       from backend.storage import get_storage
36
37       LOG = logging.getLogger("compute.cloud_run")
38
39       # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40       # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41       _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42           "deployment.compute_target=local_gpu",
43           "indexing.detector_impl=native",
44       )
45
46
47       class CloudRunJobClient(ABC):
48           """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50           Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51           overrides the job's container ``args`` for this execution and starts it."""
52
53           @abstractmethod
54           def run_job(self, job_name: str, argv: List[str], *, region: str,
55                       project: Optional[str] = None) -> str:
56               """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57               execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60       class GoogleCloudRunJobClient(CloudRunJobClient):
61           """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62           a fake, so this code path is never exercised in tests (the SDK isn't a test
```

```
dependency)."""
63
64          def run_job(self, job_name: str, argv: List[str], *, region: str,
65                      project: Optional[str] = None) -> str:
66              try:
67                  from google.cloud import run_v2  # type: ignore[attr-defined]
68              except Exception as exc:  # pragma: no cover - real SDK not present in CI
69                  raise RuntimeError(
70                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                      "control plane (it is intentionally NOT a CI/test dependency)."
72                  ) from exc
73              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
74              name = client.job_path(project, region, job_name)
75              overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77              )
78              op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79              return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82      class CloudRunCompute(ComputeBackend):
83          """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85          def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                       project: Optional[str] = None, storage_config: Optional[dict] = None,
87                       policy: ExecutionPolicy = DEFAULT_POLICY,
88                       token: Optional[str] = None,
89                       container_overrides: Optional[tuple[str, ...]] = None) -> None:
90              self._client = run_client
91              self._job_name = job_name
92              self._region = region
93              self._project = project
94              self._storage_config = storage_config or {}
95              self._policy = policy
96              # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97              # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98              self._token = token
99              self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                                "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                              project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                      self._job_name, execution, done_uri)
```

```
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128                 rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156
```

6. assistant (2026-06-22T14:45:16.170Z)

All three files read fully. Here is the documented contract.

`backend/compute/` contract (COMPUTE_DECOUPLED_SERVING M6)

All file refs are absolute paths under `C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/`.

`ComputeJobSpec` ? `base.py:20-30`
Frozen dataclass, one inference job, profile-agnostic.
- `video_uri: str` (`base.py:26`) ? input; `gs://?` for cloud, a path / `local://` for local.
- `out_uri: str` (`base.py:27`) ? output ROOT; the `gs://?` bucket for cloud.
- `segmenter: Optional[str] = None` (`base.py:28`).

- `config_overrides: tuple[str, ...] = ()` (`base.py:30`) ? raw worker `--set k=v` overrides; a tuple (not list/dict) so the spec stays frozen/hashable.

`video_id` is NOT a field ? it's the MD5 the worker re-computes from the same input bytes, so output always lands at `<out_uri>/outputs/<video_id>/` deterministically (`base.py:22-24`).

`ComputeResult` ? `base.py:33-43`
Frozen dataclass, terminal state of a dispatched job.
- `returncode: int` (`base.py:40`) ? mirrors the worker exit code: `0` ok / `2` validation-or-config / `3` segmenter-failure, **plus `4` = remote dispatch timeout or empty/unreadable completion marker (cloud path only)** (`base.py:36-38`).
- `outputs_uri: str = ""` (`base.py:41`) ? where `intervals.csv` / `report.json` landed.
- `video_id: str = ""` (`base.py:42`) ? known once the done-marker is read.
- `report: dict = field(default_factory=dict)` (`base.py:43`).

`ComputeBackend.run_infer()` ? `base.py:46-53`
Note: the ABC method is `run_infer(self, spec: ComputeJobSpec) -> ComputeResult` (`base.py:51-53`), NOT `dispatch()`. It "dispatches ONE inference job and returns only once its outputs are durably available (bucket-poll for cloud), so a caller treats local and cloud execution identically" (`base.py:47-49`).

The factory seam ? `get_compute_backend()` ? `__init__.py:32-61`
Signature: `get_compute_backend(config, *, run_client=None, policy=DEFAULT_POLICY, token=None) -> Optional[ComputeBackend]` (`__init__.py:32-38`).
- Default-OFF: returns `None` unless `config.deployment.compute_target == "cloud_run"` (`__init__.py:47-49`). `None` means "run the worker's in-process path, byte-identical."
- Only `"cloud_run"` returns a backend (a `CloudRunCompute`). Coordinates come from control-plane env, no secrets in config: `CLOUD_RUN_JOB` (default `"rally-worker"`), `CLOUD_RUN_REGION` (default `"asia-south1"`), `GOOGLE_CLOUD_PROJECT` (`__init__.py:54-56`).
- **Client injection point:** `client = run_client if run_client is not None else GoogleCloudRunJobClient()` (`__init__.py:57`). So a test passes `run_client=<fake>` here; production defaults to the real client. The SDK is lazy-imported only on the cloud branch (`__init__.py:52`).

Done-marker / bucket-poll mechanism ? `cloud_run.py`
The flow (documented `cloud_run.py:4-12`):

1. **Dispatch** (`cloud_run.py:103-117`): builds `argv` then calls `self._client.run_job(...)` which does NOT block. The argv:
   ```
   infer --video-uri <spec.video_uri> --out-uri <spec.out_uri> --done-uri <done_uri>
   ```
   (`cloud_run.py:107-108`), then `--segmenter` if set (`cloud_run.py:109-110`), then one `--set kv` per entry in `(*spec.config_overrides, *self._container_overrides)` (`cloud_run.py:111-112`).
   - `done_uri = f"{out_root}/_dispatch/{token}.done"` (`cloud_run.py:106`), where `out_root = spec.out_uri.rstrip("/")` (`cloud_run.py:104`) and `token = self._token or uuid.uuid4().hex` (`cloud_run.py:105`). The token keys the marker so concurrent jobs to one bucket don't collide; `None` ? fresh uuid4 per call (prod), injected ? deterministic (tests) (`cloud_run.py:96-98`).

2. **Bucket-poll** (`cloud_run.py:119-120`): `poll_until(lambda: self._read_marker(done_uri), self._policy, label="cloud-run-infer", logger=LOG)`. `poll_until` is imported from `backend.infrastructure.execution_policy` (`cloud_run.py:33`). The completion signal is a storage-backed marker object the worker writes, NOT the Cloud Run API status (D16, `cloud_run.py:6-10`); scale-to-zero cold-start is absorbed by the async poll (`cloud_run.py:10`).
   - `_read_marker(done_uri)` (`cloud_run.py:135-140`): parses the URI, returns `None` if `get_storage(ref.backend, config=self._storage_config).exists(ref)` is false ? `poll_until` keeps waiting; else returns `self._read_json(done_uri) or {}` (an empty dict still terminates the poll ? "present == done").
   - `_read_json` (`cloud_run.py:142-155`): downloads to a real local temp dst FIRST (`tempfile.mkdtemp`), because cloud backends (gcs/s3) REQUIRE a dst ? a `None` dst raises `ValueError`, unlike local's identity passthrough; without an explicit dst this read would silently return `{}` for every gs://|s3:// object (`cloud_run.py:143-145`). Returns `{}` on any

exception.

3. **Read** (`cloud_run.py:121-132`): from the marker, `video_id = str(marker.get("video_id", ""))` (`cloud_run.py:121`) and `rc = int(marker.get("returncode", 1))` ? **a missing returncode defaults to FAILURE (1), never success** (`cloud_run.py:122-124`). If `video_id` is empty ? warn + `rc = rc or 1` (`cloud_run.py:125-128`). `outputs_uri = f"{out_root}/outputs/{video_id}/"` if a video_id exists else `""` (`cloud_run.py:129`); then `report = self._read_json(f"{outputs_uri}report.json")` (`cloud_run.py:130`). Returns `ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id, report=report)` (`cloud_run.py:132`).

Real vs injectable-fake Cloud Run client ? this is how item 1 is tested with NO real GPU

`CloudRunJobClient` ABC (`cloud_run.py:47-57`): one abstract method
```
run_job(self, job_name: str, argv: List[str], *, region: str, project: Optional[str]=None) -> str
```
Starts a Cloud Run Job execution running `python -m backend.worker <argv>`, returns an execution id/name for logs, and "Does NOT block on completion (the caller bucket-polls)" (`cloud_run.py:54-57`).

`GoogleCloudRunJobClient` (real) ? `cloud_run.py:60-79`: lazy-imports `google.cloud.run_v2` (`cloud_run.py:67`); raises `RuntimeError` if the SDK is absent (`cloud_run.py:68-72`, "intentionally NOT a CI/test dependency"). Builds `JobsClient()` ? `job_path(project, region, job_name)` ? a `RunJobRequest.Overrides` with a single `ContainerOverride(args=argv)` ? `client.run_job(...)`. Every real-SDK line is marked `# pragma: no cover` ? "CI uses a fake, so this code path is never exercised in tests" (`cloud_run.py:61-62`). Owner-verified on the M7 real run.

How the fake is injected (no GCP / no GPU): `CloudRunCompute.__init__` takes `run_client: CloudRunJobClient` as a required keyword-only arg (`cloud_run.py:85-90`) and stores it as `self._client` (`cloud_run.py:90`). A test subclasses `CloudRunJobClient` with a fake whose `run_job` (a) records the argv and (b) ? in the same call or via a paired storage stub ? writes the done-marker JSON (`{"video_id": ..., "returncode": ...}`) to `done_uri` in the configured storage so `_read_marker` finds it and `poll_until` terminates. The fake reaches `CloudRunCompute` either directly via `CloudRunCompute(run_client=fake, ?)` or through the factory via `get_compute_backend(config, run_client=fake)` (`__init__.py:35,57`). The `token` kwarg makes `done_uri` deterministic for assertions (`cloud_run.py:96-98,105-106`). Because the storage layer (`backend.storage`) already has a `local`/in-memory backend, the entire dispatch?marker?poll?read?`ComputeResult` cycle runs offline with no `google-cloud-run`, no Cloud Run, and no GPU.

What `CloudRunCompute` assumes about the deployed Cloud Run Job image

- **The container runs the SAME worker entrypoint** `python -m backend.worker` and the client only overrides its `args` per execution (`cloud_run.py:5,48-51,54-57,76-78`). The image's baked entrypoint must therefore be `python -m backend.worker` (the override supplies only argv, e.g. `["infer", "--video-uri", ?]`).
- **The worker CLI must accept these flags:** `infer`, `--video-uri`, `--out-uri`, `--done-uri`, `--segmenter` (optional), and repeatable `--set k=v` (`cloud_run.py:107-112`).
- **The worker must write the done-marker** to `--done-uri` as a JSON object carrying `video_id` and `returncode` (both always written ? `cloud_run.py:38,121-124`), and write `report.json` (and `intervals.csv`) under `<out_uri>/outputs/<video_id>/` (`cloud_run.py:11-12,129-130`; `base.py:22-24,38`).
- **Forced INLINE execution to prevent recursive re-dispatch** via the default container overrides `_DEFAULT_CONTAINER_OVERRIDES` (`cloud_run.py:41-44`), passed as `--set`:
  - `deployment.compute_target=local_gpu` ? disarms the `cmd_infer` dispatch guard so the container computes instead of re-dispatching.
  - `indexing.detector_impl=native` ? the L4 WASB runner.

  These are appended after `spec.config_overrides` (`cloud_run.py:111`) and are overridable via the `container_overrides` ctor kwarg (`cloud_run.py:89,99-101`).
- **Env-supplied coordinates** (from the factory, not the image): `CLOUD_RUN_JOB`, `CLOUD_RUN_REGION`, `GOOGLE_CLOUD_PROJECT` (`__init__.py:54-56`) ? these name the deployed Job, its region, and project.
- **Storage assumption:** the worker writes to, and the dispatcher reads from, the same storage

bucket via `backend.storage` using `self._storage_config` (passed from `config.storage` via `_storage_dict`, `__init__.py:23-29,60`); the marker and outputs are `gs://`/`s3://` objects, which is why `_read_json` forces a local dst (`cloud_run.py:143-149`).

One observation worth flagging: the module docstring (`cloud_run.py:1`) and `base.py:47` say the dispatcher "bucket-polls", but `ComputeBackend`'s concrete method is `run_infer`, while `__init__.py`'s docstring at line 45 references "the real `google-cloud-run` client". There is no method literally named `dispatch()` anywhere ? the dispatch step is the internal `self._client.run_job(...)` call inside `run_infer` (`cloud_run.py:114`). If your task spec refers to a `dispatch()` method, it does not exist by that name; the public entrypoint is `ComputeBackend.run_infer(spec)`.